

Syllabus Summary

A hands-on robotics project where students build and control a robotic arm using a game controller and ROS2 on the QNX Real Time Operating System on a Raspberry Pi. The project teaches robotics fundamentals, embedded systems, and periodic 50 Hz control through practical experimentation. It introduces how a real-time operating system supports responsive embedded applications.



Project Overview

This project demonstrates how to control a 5-degree-of-freedom robotic arm, plus a separately controlled gripper, using a gamepad and ROS2 software on QNX. Five articulated joints position and orient the arm, while the sixth servo opens and closes the gripper.

Learning Goals

After completing this project, students should be able to:

- Understand basic robotics concepts (joints, motion, velocity)
- Use ROS2 to build simple robotic applications
- Understand basic QNX concepts and commands
- Connect typical embedded hardware (servos, gamepad, Raspberry Pi)
- Read and modify Python or C++ robotics code
- Understand periodic control loops and how QNX supports real-time systems
- Map user input to physical motion



Hands-On Teaching Module

Baby Robot Arm with Gamepad Control

Target Student Level

- **High school** – Suitable for advanced students if strong guidance is provided
- **Undergrad year 1-2** – Ideal target student
- **Undergrad year 3-4** – Suitable if extend the sample to include other challenges
- **Graduate** – Maybe, depending on the use case. Could be good for grad students supervising undergrad project creation in robotics labs, using as a base project to explore AI/ML techniques with computer vision or explore digital twins, etc.

Prerequisites (Required Skills)

Students should have:

- **Basic programming.** Familiarity with Python is recommended.
- **Basic Linux command line skills.** QNX is the operating system used for the project, but because QNX is POSIX compliant, it is very similar to Linux. Students having some facility at the Linux command-line will easily adapt to QNX commands.
- **Intro to electronics (wiring, power).** Students should be comfortable about handling circuit boards and have appropriate electronics discipline (careful around static discharge, not placing boards on metal surfaces, etc). No soldering is required.

The following skills are helpful but optional, although they can be learned through the process of using the project:

- **Basic concepts of robotics are helpful but not necessary.** Concepts of kinematics and 3D space are helpful to understand how the robot works and is controlled but can be taught using the robot as an example.
- **C++ experience.** In addition to Python, C/C++ is also used in project, which could provide an opportunity to contrast the difference between “hard” vs “soft” programming languages used in embedded projects.
- **ROS2 familiarity.** The program uses three ROS2 nodes for gamepad input, inverse kinematics, and arm control. If the student has no ROS exposure, they can quickly develop an understanding of essential ROS concepts using the robot arm source as a guide.
- **Embedded systems exposure.** Understanding concepts like host/target development, image creation, networking, and file transfer are useful to get the students building software on their laptops and transferring it to the Raspberry Pi.



Hands-On Teaching Module

Baby Robot Arm with Gamepad Control

Hardware Requirements

Minimum setup per student/team:

- Raspberry Pi 4B or Raspberry Pi 5
- 5-DOF 3D-printed robotic arm kit with a separate gripper
- PCA9685 servo driver (I²C based)
- Logitech F310 or F710 gamepad in XInput or DirectInput mode; validate captured reports from the exact controller and QNX HIDI stack before enabling servo power
- 6V power supply for servos
- Six servos: three 270-degree high-torque servos for joints 0-2 and three 180-degree micro servos for joints 3-5, including the gripper
- Wiring, a guarded workspace, a physical emergency stop, a normally-off PCA9685 OE or servo-power watchdog, and a verified physical wrist-channel order

Optional:

- Camera (for vision-based extensions)

Also, a 3D printer is necessary to print custom robot arm parts.

Software Requirements

All software is provided or free to download.

- Robot arm specific code and instructions: <https://github.com/qnx/Baby-Robot-Arm-with-Gamepad-Control>
- Free version of [QNX SDP 8](#) (includes Python and C++ toolchains and build tools)
- ROS2 Jazzy for QNX aarch64le, installed under /opt/ros/jazzy
- Python runtime modules: rclpy and smbus
- C++ and ROS dependencies: rclcpp, sensor_msgs, std_msgs, builtin_interfaces, rcl_interfaces, ament_index_cpp, URDF/KDL, kdl_parser, Eigen3, TinyXML2, the QNX HIDI library, and Screen development headers
- A dedicated unprivileged service account with only the required HIDI and I2C permissions; a root-owned promoted install; and deny-by-default SROS2 identities for each active node



Hands-On Teaching Module

Baby Robot Arm with Gamepad Control

System Architecture (Simple Explanation)

The repository contains three ROS2 nodes. The secured default launch runs the gamepad and arm-controller nodes with autonomous demonstrations disabled; inverse kinematics is experimental and opt-in:

1. Gamepad Input Node (C++)

- o Accepts only length- and framing-validated Logitech F310/F710 gamepad reports through QNX HIDI and attaches exclusively to one active device
- o Publishes relative joy messages at up to 50 Hz; controller loss, an invalid report, or a 250 ms timeout sends a final neutral and exits nonzero so supervision stops the control unit

2. Arm Controller (Python)

- o Starts disarmed and accepts bounded, finite, fresh commands only while L3 is held; production arming requires a bound hardware OE or servo-power hook
- o After hardware-hook integration, hold L3 and press the exact neutral SELECT+START+A chord to arm; releasing L3 disables output, and calibrated limits and slew bounds remain active
- o Inside a per-robot SROS2 namespace, IK topics are relative; CurrentPositions and CartesianCmd carry a bounded session epoch, but CurrentPositions is a commanded open-loop estimate, not measured feedback

3. Inverse Kinematics Solver (C++)

- o Uses Orocos KDL LMA only after a confined model passes an exact five-joint, bounded-chain validation contract
- o Rejects malformed/non-finite inputs, mismatched or stale session epochs, excessive callback gaps, and discontinuous solutions before publishing exact, ordered five-joint commands tagged to that epoch on relative mov
- o The checked-in URDF intentionally fails validation. Keep IK disabled until a measured five-joint model, tool-center point, collision checks, and verified homing or position feedback are supplied

Estimated Project Time

Activity	Time
Hardware assembly	3–6 hours
Software setup	2–4 hours
Running demo	1 hour
Understanding system	3–5 hours
Extensions/project	1–4 weeks



Hands-On Teaching Module

Baby Robot Arm with Gamepad Control

Note: If students print all mechanical parts themselves, expect an additional 20–50 hours of total printing time (approximately 1 week of elapsed time depending on printer availability). Assuming they're familiar with 3D printing software and equipment, the hands-on student effort to manage the printing is typically 5–9 hours, which includes setup and post-processing.

Suggested Course Integration

Option A: Short Module (1–2 weeks)

- Intro to robotics + ROS2
- Build and run system
- Simple modifications

Option B: Lab Series (3–6 weeks)

- Week 1: Build hardware
- Week 2: Understand control system
- Week 3: Modify mappings
- Week 4+: Add features

Option C: Full Project (term)

- Students extend functionality
- Present final demos

Example Student Activities

Beginner:

- Change joystick mapping
- Adjust speed of arm movement
- Test calibrated servo limits, stale-input watchdogs, and fault-triggered output disable with servo power disconnected

Intermediate:

- Compare and extend the existing joystick, direct-joint, and IK control modes
- Implement recording/playback of motions
- Tune servo parameters

Advanced:

- Add computer vision (camera tracking)
- Evaluate or extend the existing KDL inverse kinematics solver
- Design bounded autonomous behaviors only after verifying the physical joint map; retain explicit arming, an expiring command lease, and a hardware output-disable watchdog
- Evaluate a web or AI command gateway only after adding authenticated authorization, schema validation, rate limits, and independent actuator bounds



Hands-On Teaching Module

Baby Robot Arm with Gamepad Control

Extension/Expansion:

- AI/ML: Train model to view and imitate user motion
- Human-computer interaction: alternative control methods
- Real-time systems: measure 50 Hz loop latency, jitter, and responsiveness
- Embedded systems: low-level hardware interfacing
- Robotics: motion planning, kinematics

Instructor FAQ

Q: Is this beginner-friendly?

Yes, with guidance; students without embedded-systems experience need support.

Q: Can this project be used without QNX?

It has not been tested on Linux; the code and instructions target QNX, although the concepts transfer.

Q: Do you need a 3D printer?

Currently yes, unless an instructor supplies preprinted parts or a complete kit.

Q: Is this project safe?

No software-only configuration makes an exposed robot arm inherently safe. The secured profile refuses to arm until a board-specific OE or servo-power hook is integrated. Use instructor supervision, a guarded clear zone, calibrated limits, and a physical emergency stop. Commission with servo power disabled first; a fault must disable output rather than move to a home pose.